**Intelligent and Communicating Systems, ICS**

$2^{nd}$ Year Specialty SIQ G02

# LAB report n°06

# Iot System based Platform Cloud and standalone

**Studied by:**

**First Name: Amel**
**Last Name: Sadoun**
**E-mail: ma_sadoun@esi.dz**

# Chapter 1

# A. Theory

There are many IoT Cloud or Standalone platforms that target similar use cases as Ewelink, including Blynk, Arduino Cloud, SmartThings, Openhab, MyOpenhab, Home Assistant, Jeedom, etc. These platforms aim to provide user-friendly experiences for home automation tasks and generally offer mobile apps for easy control and monitoring of connected devices. However, each platform may have unique features, pricing, open-source status, and compatibility with different IoT devices. Evaluating them is essential to choose the one that best fits specific needs and requirements.

## 1.1 Comparison of Four Well-Known IoT Platforms

Four widely used platforms are:

- **Blynk:** Cloud-based platform focused on mobile IoT control. Compatible with Arduino, ESP, and Raspberry Pi. Proprietary software, free tier available, paid plans for advanced features. User-friendly mobile app for monitoring and automation.
- **Arduino Cloud:** Official Arduino IoT platform. Supports Arduino boards, partially open-source libraries. Subscription-based for advanced functionalities. Ideal for Arduino-centric IoT projects.
- **Home Assistant:** Open-source platform for local or cloud deployment. Supports thousands of devices and automation rules. Free, highly customizable, suitable for advanced home automation setups.
- **SmartThings:** Samsung's IoT platform. Cloud-based, proprietary software, supports Zigbee/Z-Wave devices. Free for basic use; subscription for advanced features. Focused on consumer smart home automation.

| Platform | Use Case | Open Source | Price / Notes |
|---|---|---|---|
| Blynk | Mobile IoT control | No | Free limited, Paid Pro |
| Arduino Cloud | Arduino boards | Partial | Free + Paid tiers |
| Home Assistant | Home automation | Yes | Free |
| SmartThings | Home automation | No | Free basic, Paid advanced |

Table 1.1: Comparison of 4 IoT Platforms

## 1.2 Cloud vs Standalone (On-Premise) IoT Platforms

Cloud IoT platforms and standalone (on-premise) platforms differ mainly in hosting, control, scalability, and cost. According to industry analyses, cloud platforms host infrastructure and services remotely, offering scalability and ease of use, whereas on-premise platforms run locally, providing greater data control and compliance options.

| Feature | Cloud IoT | Standalone (On-Premise) |
|---|---|---|
| Hosting | Remote servers, vendor-managed | Local servers, self-managed |
| Data Control | Limited direct control | Full control and sovereignty |
| Security | Depends on provider's measures | Can be stronger due to isolation |
| Scalability | Easily scales as needed | Harder and costly to scale |
| Deployment Cost | Low initial cost | High upfront hardware cost |
| Maintenance | Provider handles updates | Requires internal IT team |
| Use Case | Consumer IoT, rapid deployment | Sensitive data, industrial IoT |

Table 1.2: Comparison of Cloud vs Standalone IoT Platforms

Cloud IoT platforms allow rapid deployment, lower initial costs, and remote access from anywhere, benefiting applications that need quick scaling and mobile dashboards. On the other hand, standalone platforms give complete data control and can satisfy strict compliance or data residency requirements typical in industrial, medical, or regulated environments.

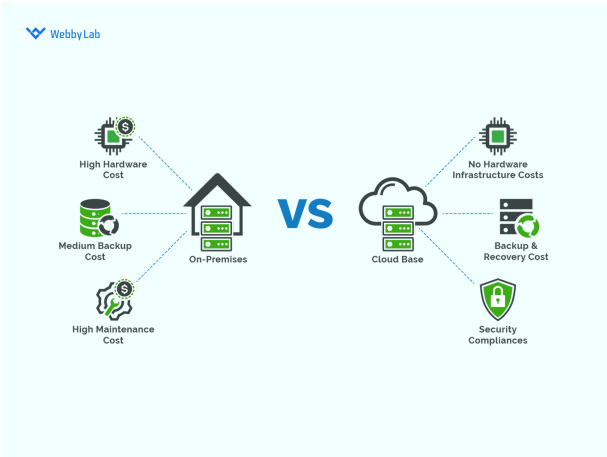


Figure 1.1: Example IoT platform architecture: Cloud vs Local deployment

# Chapter 2

# B. Activity

## 2.1 IoT System Setup: Arduino LED Controlled via Mobile App

In this activity, a simple IoT system was considered where an Arduino board controls a single LED. The control is entirely virtual: a push button on the mobile app acts as the input, while the Arduino board only hosts the LED and its current-limiting resistor. The system is connected to cloud-based or standalone IoT platforms to monitor and control the LED remotely.

### 2.1.1 Hardware Configuration

- Arduino board (e.g., MKR1010)
- LED connected to digital pin D2 with a resistor
- Power via USB

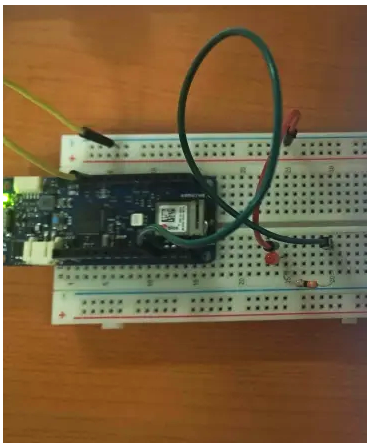

Figure 2.1: Arduino hardware setup with LED and resistor

### 2.1.2 Software Configuration

The Arduino board runs minimal code to receive the LED state from the IoT platform. The mobile app push button sends commands to the platform (Blynk or Arduino Cloud), which then updates the LED state on the board.

## 2.2 Cloud-Based Platforms Experiments

### 2.2.1 Blynk Platform

Blynk is a popular IoT platform that allows you to build mobile and web interfaces to control hardware devices remotely. It supports multiple boards, including Arduino, and can interact with sensors, LEDs, and other actuators in real time. In this activity, we used Blynk to control an LED connected to an Arduino MKR WiFi 1010 board.

The following steps describe the creation process:

- Create a new project in the Blynk mobile app.

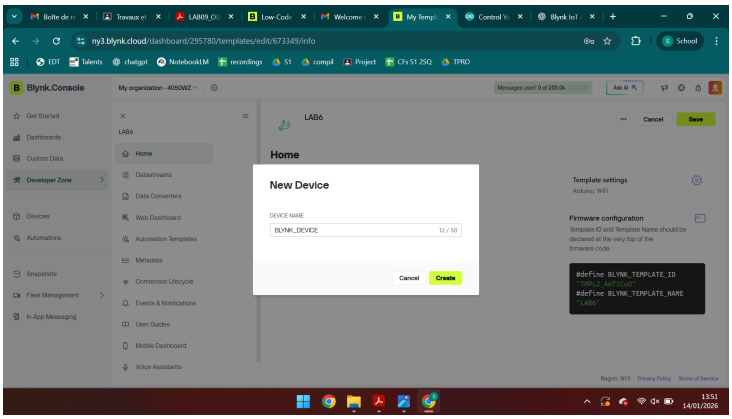


Figure 2.2: Blynk: Creating the device in the mobile app

- Select the device: Arduino MKR WiFi 1010.
- Connect the device to Wi-Fi by entering the SSID and password.
- Add a data stream (widget) to control the LED. In this case, a Button widget was assigned to Virtual Pin V0 (Digital Pin 2).

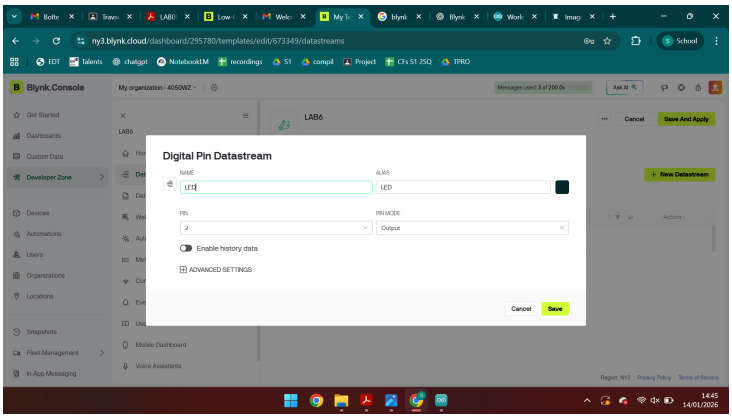


Figure 2.3: Blynk: Adding a data stream (Button widget for Digital Pin 2) for LED control

- Upload the Arduino sketch with the Blynk authentication token to the MKR WiFi 1010 board.

**Arduino Code for Blynk LED Control**

```
// Blynk template settings
#define BLYNK_TEMPLATE_ID "TMPL2_AeT1CuO"
#define BLYNK_TEMPLATE_NAME "LAB6"
#define BLYNK_PRINT Serial
#include <WiFiNINA.h>
#include <BlynkSimpleWiFiNINA.h>
#define LED_PIN 2  // external LED

char auth[] = "WFOcRV-POATutBniPOIf55kbaUNYWC3d";
char ssid[] = "hopingwme";
char pass[] = "amelissupercool";

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
  Blynk.begin(auth, ssid, pass);
}

BLYNK_WRITE(V0) {  // V0 = button widget in your Blynk app
  int buttonState = param.asInt();  // 0 = OFF, 1 = ON
  digitalWrite(LED_PIN, buttonState);

  // Optional: update LED widget in app
  Blynk.virtualWrite(V1, buttonState);
}

void loop() {
  Blynk.run();
}
```

- After downloading the Blynk mobile app and logging in with the same account as the website, we can find the device.

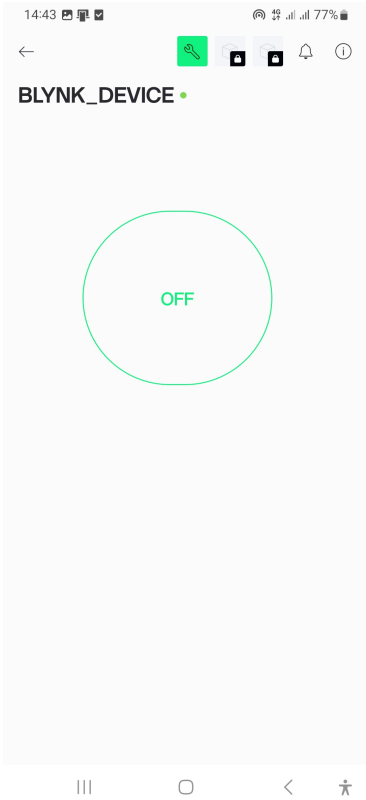


Figure 2.4: Blynk mobile app button interface

- Test the setup: pressing the mobile app button should turn the LED ON or OFF in real time.

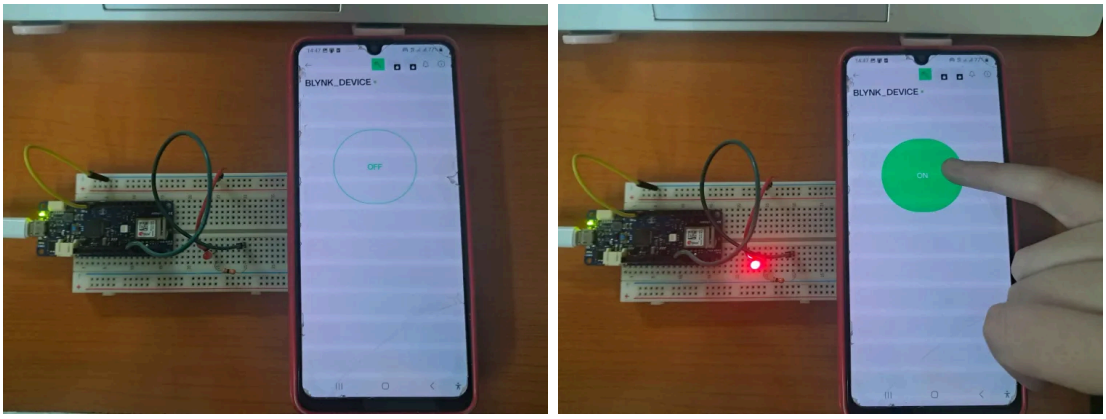


Figure 2.5: Blynk LED control through phone (hardware + phone app interface)

- The device can also be controlled through the web interface.

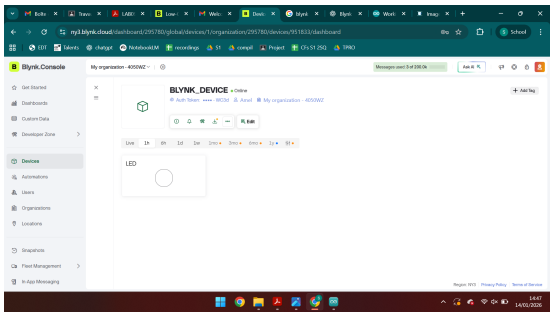


Figure 2.6: Blynk device web page (OFF)

### 2.2.2 Arduino Cloud Platform

Arduino Cloud is an IoT platform provided by Arduino that allows remote monitoring and control of devices. It provides dashboards, cloud properties, and mobile/web interfaces to interact with connected hardware. In this activity, we used Arduino Cloud to control an LED connected to an Arduino MKR WiFi 1010 board.

The following steps describe the setup and usage process:

1. Create a new device on the Arduino Cloud website and link it to your MKR WiFi 1010 board.
2. Add a cloud property to represent the LED state. In this case, the property is named ‘light‘ and is of type ‘Boolean‘. Assign this property to control Digital Pin 2 on the Arduino board.

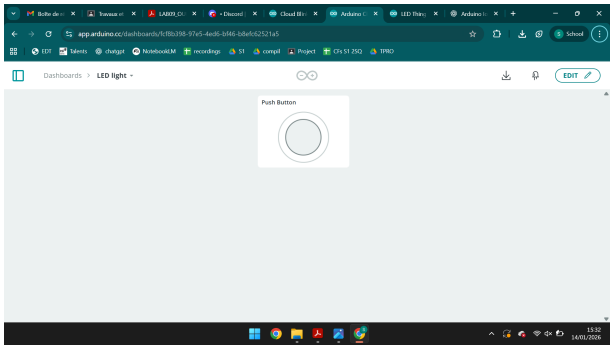


Figure 2.7: Arduino Cloud: dashboard button for LED control

3. Upload the Arduino sketch to the online board. Here is the main Arduino Cloud code:

**Arduino Cloud LED Control Code**

```
#include "thingProperties.h" // Include generated file

const int LED_PIN = 2; // External LED pin

void setup() {
  Serial.begin(9600);

  // Setup LED pin
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  // Start cloud connection
  initProperties();
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);
}

void loop() {
  ArduinoCloud.update(); // Keep cloud connection alive
}

// Called whenever cloud property changes
void onLightChange() {
  digitalWrite(LED_PIN, light ? HIGH : LOW); // Turn LED on/off
}
```

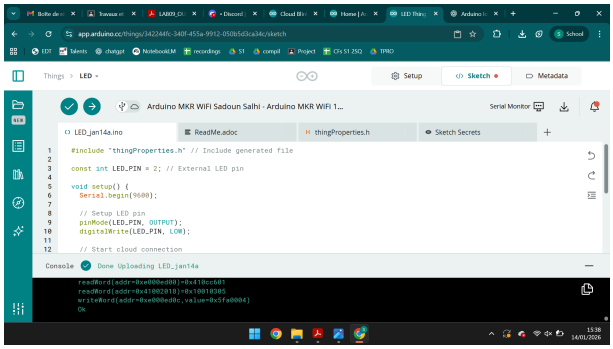


Figure 2.8: Arduino Cloud: Code page

4. After setting up the cloud property, open the Arduino Cloud mobile app to locate your device.

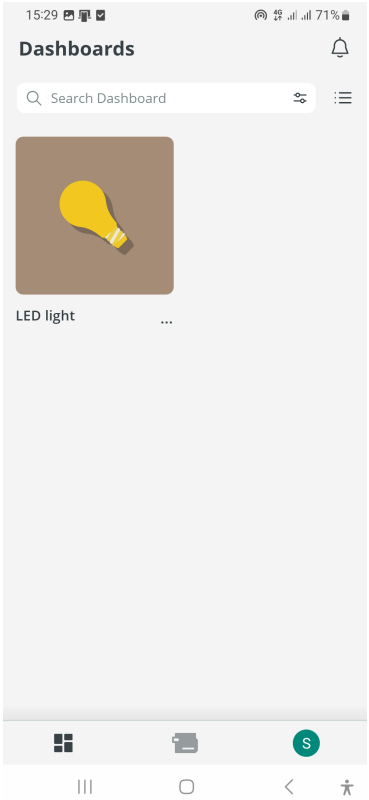


Figure 2.9: Arduino Cloud: Mobile app showing the LED device and control button

5. Test the LED control via the mobile app. Pressing the toggle button turns the LED ON or OFF in real time.

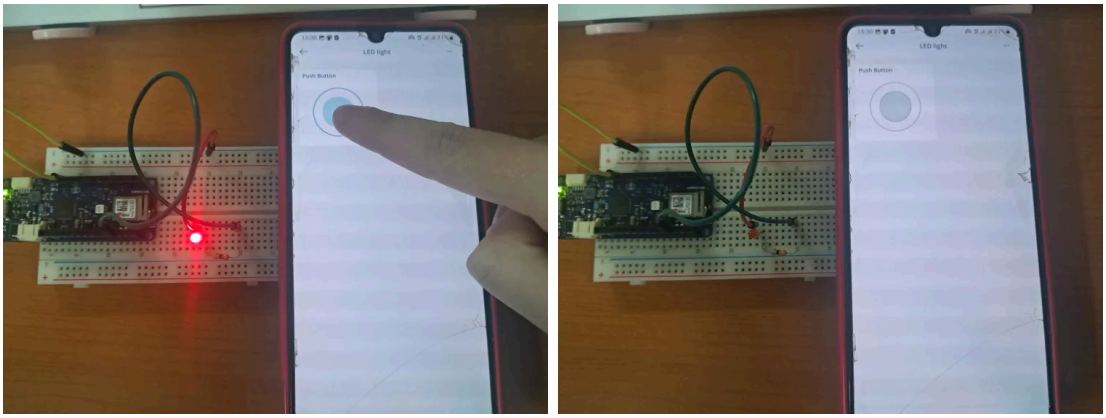

Figure 2.10: Arduino Cloud: Mobile app controlling LED ON and OFF (hardware + app interface)

6. The LED can also be controlled via the Arduino Cloud web dashboard. Pressing the dashboard button updates the LED state.

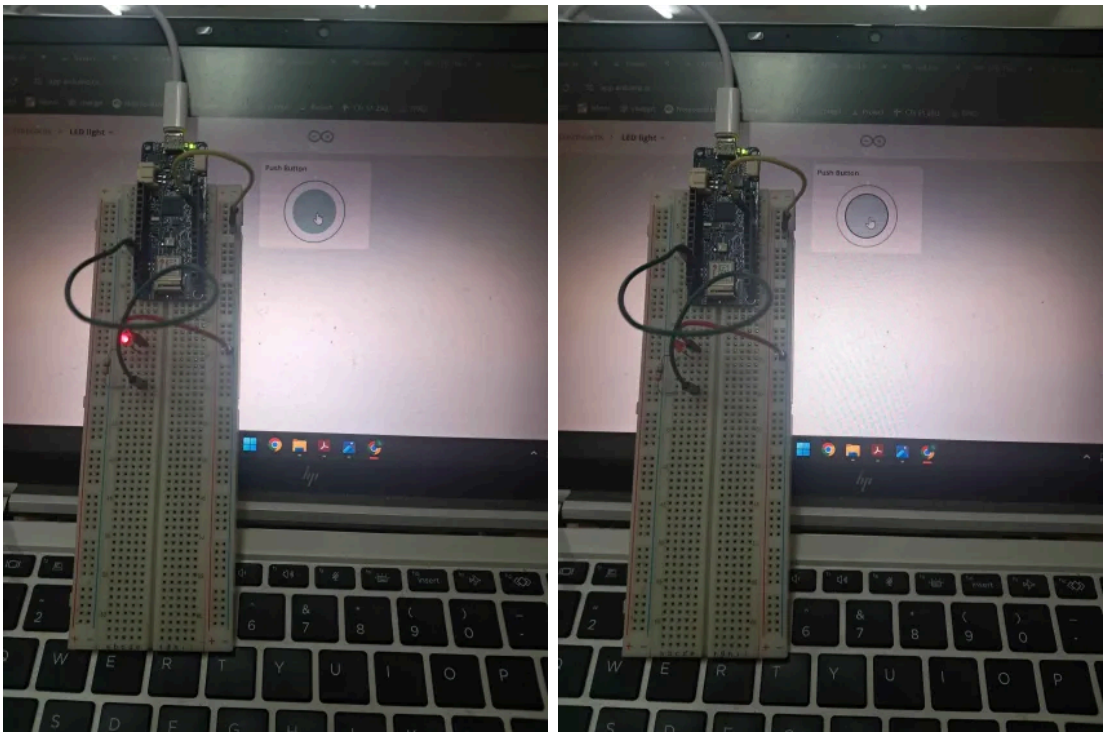

Figure 2.11: Arduino Cloud: Web dashboard controlling LED ON and OFF

### 2.2.3 Internet Requirement

For cloud-based platforms, the internet connection is required constantly for the LED to reflect the state sent by the phone app. Once the command is sent, the platform relays the state to the Arduino board in real time. During our tests, when Wi-Fi and mobile data were turned off on the phone, the LED could not be controlled, demonstrating that continuous internet connectivity is necessary for cloud-based operation.

### 2.2.4 Standalone Platform: Home Assistant (Local Control)

Home Assistant was installed on a Raspberry Pi to enable local control of the LED without relying on the internet. By using the official LAN mode or LocalTuya for Tuya devices, the LED can be controlled entirely over the local network.

The steps followed were:

- Access the Raspberry Pi via SSH and check the Home Assistant Docker container:

```
ssh pi@192.168.137.2
docker ps        # Check if Home Assistant container is running
docker start homeassistant    # Start container if not running
```

- Open a web browser on your laptop connected to the same network and navigate to the local Home Assistant dashboard:

```
http://192.168.137.2:8123
```

Log in with your credentials to access the dashboard.

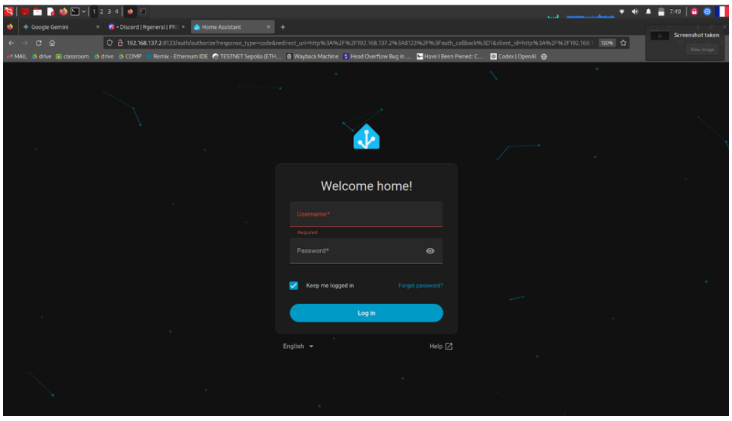


Figure 2.12: Home Assistant dashboard on Raspberry Pi

- Install LocalTuya manually. On your laptop:

```
git clone https://github.com/rospogrigio/localtuya.git
```

Copy the folder to the Raspberry Pi (assuming Pi IP is 192.168.137.2):

```
scp -r localtuya pi@192.168.137.2:~/
```

On the Raspberry Pi, move the folder into Home Assistant's 'custom$_{c}omponents$'$folder$ :

```
mkdir -p ~/homeassistant/custom_components
mv ~/localtuya ~/homeassistant/custom_components/
docker restart homeassistant    # Restart container to load LocalTuya
```

Figure 2.13: LocalTuya folder placed in Home Assistant custom$_{c}omponents$

- Retrieve the Tuya local keys once via Tuya Cloud. Steps:

- Create a Tuya Developer account at https://iot.tuya.com.
- Create a new Cloud Project (e.g., “Home Assistant”) and select the correct Data Center.
- Obtain the Client ID and Secret from the Authorization Key section.
- Link your Tuya mobile app account to the cloud project and copy the User ID (UID).
- Enter these credentials in Home Assistant LocalTuya integration to fetch the local keys.

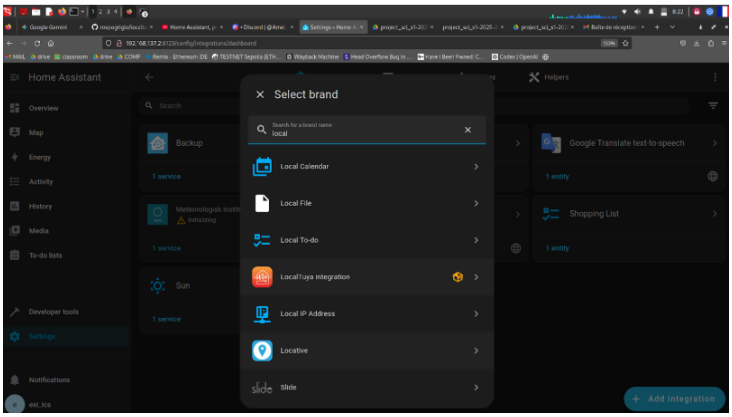


Figure 2.14: LocalTuya setup screen in Home Assistant

- After completing these steps, the LED can be controlled locally from the Home Assistant dashboard, even with Wi-Fi and internet turned off.

## 2.3 CONCLUSION

The experiments show that:

- Cloud platforms (Blynk, Arduino Cloud) provide easy remote access, but require constant internet connectivity.
- Mobile app buttons can replace physical inputs, simplifying hardware setup.
- Standalone platforms can run locally on Raspberry Pi, allowing LED control without relying on the internet, improving privacy and reliability.
- For small IoT systems with limited devices, both cloud and standalone approaches are feasible, depending on connectivity and scalability requirements.